MACHINE LEARNING OPERATIONS · AIMLCZG523

Assignment 01

Heart Disease Risk Prediction

An end-to-end MLOps pipeline: from UCI data to a monitored Kubernetes API

Student	Suchendra Sharma
BITS ID	2025ae05515
Email	2025ae05515@wilp.bits-pilani.ac.in
Date	19 October 2026
Repository	https://github.com/suchendrasharma/-MLOps-AIMLCZG523_Assignment-01

At a glance

- **Champion model:** Logistic Regression (C = 0.1, L2), CV ROC-AUC **0.912 ± 0.017**, hold-out ROC-AUC **0.962**, recall **0.929**.
- **Tracking:** 357 MLflow runs (every grid candidate as a nested run), champion registered as `heart-disease-classifier@champion`.
- **Quality:** 131 automated tests; CI/CD pipeline lint → test → train + quality gate → docker on every push (about 5 minutes).
- **Serving:** FastAPI in a non-root Docker image, 2 replicas on Kubernetes behind a LoadBalancer, Prometheus + Grafana monitoring with 6 alerts including prediction drift.

1 Overview and requirement mapping

The task is to predict the presence of heart disease from 13 routinely collected clinical features and to deliver the model as a reproducible, tested, containerised and monitored API. The solution uses the UCI Heart Disease (Cleveland) data, a scikit-learn pipeline tracked in MLflow, a FastAPI service packaged with Docker, a GitHub Actions pipeline, a Kubernetes deployment on Docker Desktop and a Prometheus/Grafana monitoring stack. Every number in this report is generated from the repository's own artefacts (`reports/build/facts.json`).

Task (marks)	What was built	Where
1 Data & EDA (5)	Download script with fallback, cleaning + code-book validation, 6 EDA figures with written findings	§4
2 Features & models (8)	ColumnTransformer pipeline, engineered feature ablation, 3 model families tuned with stratified 5-fold GridSearchCV, documented selection rule	§5
3 Experiment tracking (5)	MLflow: params, CV + test metrics, plots, CV tables and models for all 357 runs; model registry with alias	§6
4 Packaging & reproducibility (7)	joblib pipeline + MLflow pyfunc model + metadata card, pinned requirements, seeds, reproducibility tests	§7
5 CI/CD & testing (8)	131 pytest tests; GitHub Actions: lint, test (coverage gate), train + quality gate, artefacts per run	§9
6 Containerisation (5)	Multi-stage non-root image, /predict returns prediction + confidence, smoke-tested locally and in CI	§8
7 Deployment (7)	Kubernetes manifests (kustomize), LoadBalancer, probes, PDB; verified load balancing, self-healing, zero-downtime rollout	§10
8 Monitoring & logging (3)	JSON request logs, Prometheus metrics + 6 alerts, 12-panel Grafana dashboard	§11
9 Documentation (2)	This report, README, architecture diagram, screenshots	this report, README

Table 1. *Assignment tasks mapped to the implementation and the section that documents it.*

2 Architecture

The system has four layers. Data flows left to right within a layer and downwards between layers: the cleaned dataset feeds training, the trained artefact feeds the CI image build, and the image runs in the cluster where it is monitored.

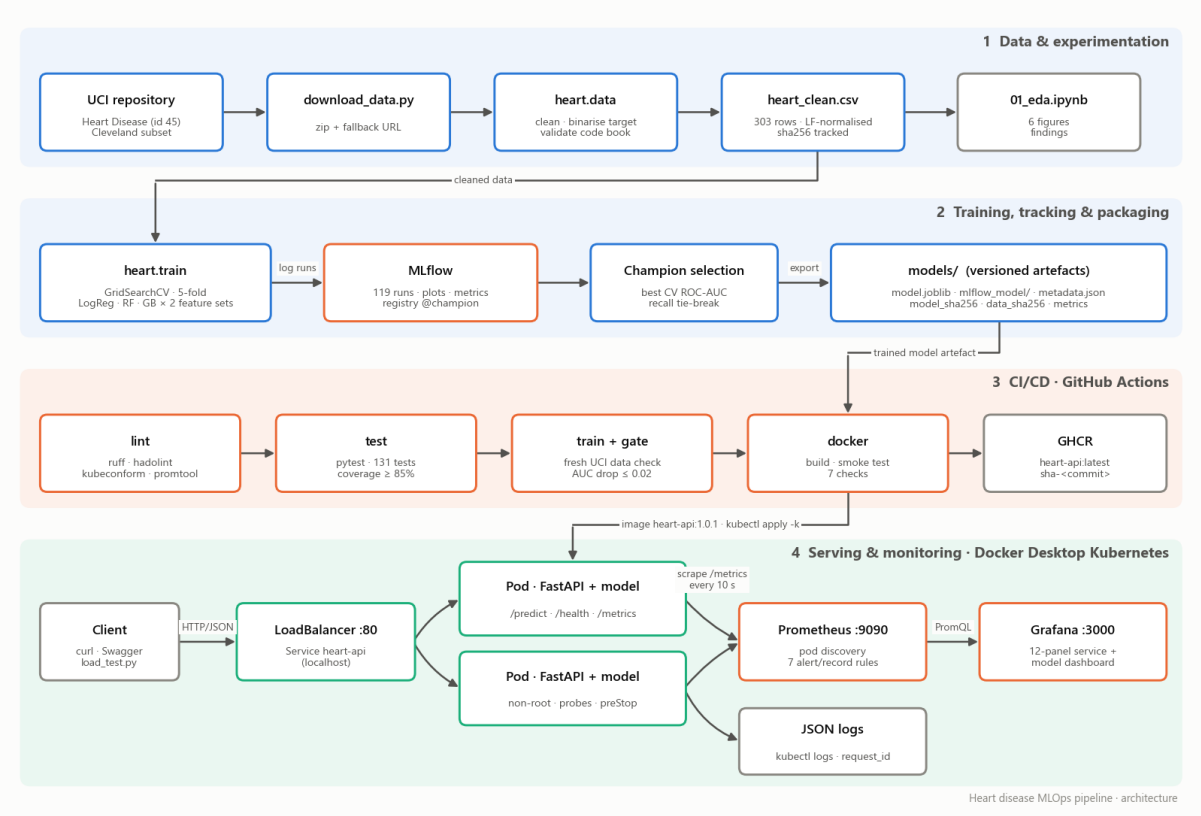


Figure 1. *System architecture: data, training and tracking, CI/CD, and serving with monitoring.*

Concern	Choice
Language / ML	Python 3.11, pandas 2.2, scikit-learn 1.5 (pinned)
Experiment tracking	MLflow 2.17 (SQLite backend + local artifact store, model registry)
Serving	FastAPI + Pydantic + Uvicorn; prometheus-fastapi-instrumentator; python-json-logger
Testing / lint	pytest + pytest-cov, ruff, hadolint, kubeconform, promtool, actionlint
CI/CD	GitHub Actions; images pushed to GitHub Container Registry
Runtime	Docker (multi-stage, non-root), Kubernetes 1.34 on Docker Desktop (kind), kustomize
Monitoring	Prometheus 2.54 (pod discovery, alert rules), Grafana 11.2 (provisioned dashboard)

Table 2. *Technology stack.*

3 Setup and installation

Prerequisites: Python 3.11, Docker Desktop with Kubernetes enabled (context `docker-desktop`), git. All commands are run from the repository root (Windows PowerShell shown; the same commands work in bash).

```
git clone https://github.com/suchendrasharma/-MLOps-AIMLCZG523_Assignment-01
py -3.11 -m venv .venv ; .\.venv\Scripts\Activate.ps1
pip install -r requirements-dev.txt ; pip install -e .
python scripts/download_data.py          # UCI Cleveland file -> data/raw/
python -m heart.data                     # clean + validate ->
data/processed/heart_clean.csv
python -m heart.train                    # grid search, MLflow tracking, export to models/
pytest                                   # 131 tests
mlflow ui --backend-store-uri sqlite:///mlflow.db        # http://127.0.0.1:5000
docker build -t heart-api:local . ; docker run -p 8000:8000 heart-api:local
.\scripts\k8s_deploy.ps1 -Monitoring       # build, deploy to Kubernetes, smoke test
python scripts/load_test.py --duration 300 # traffic for the dashboards
```

After deployment the API is available at **http://localhost** (`/predict`, `/predict/batch`, `/health`, `/metrics`, `/docs`), Prometheus at **http://localhost:9090** and Grafana at **http://localhost:3000** (dashboard opens as the home page).

4 Data acquisition and exploratory data analysis

4.1 Acquisition and cleaning

`scripts/download_data.py` downloads the official UCI archive (dataset 45) and extracts `processed.cleveland.data`, falling back to the legacy direct-file URL. The Cleveland subset (303 patients, 13 features) was chosen over the full 920-row multi-site data because the Hungarian, Swiss and VA subsets are largely missing `ca` and `thal`, two of the strongest predictors. CI re-downloads the data on every run and fails if the cleaned CSV is not byte-identical to the committed copy.

- `?` markers are parsed as missing values: only **6 cells** (`ca` 4, `thal` 2). They are kept as NaN in the cleaned CSV and imputed **inside the model pipeline**, so imputation statistics come from training folds only.
- The severity target `num` (0–4) is binarised: 0 = no disease, 1–4 = disease. Classes are close to balanced (**45.9%** positive), so stratified splits are used without resampling.
- Every categorical code is validated against the UCI code book (for example `cp ∈ {1,2,3,4}`, `thal ∈ {3,6,7}`); duplicates are dropped; the CSV is written with LF line endings so its hash is identical on every OS.

4.2 Findings

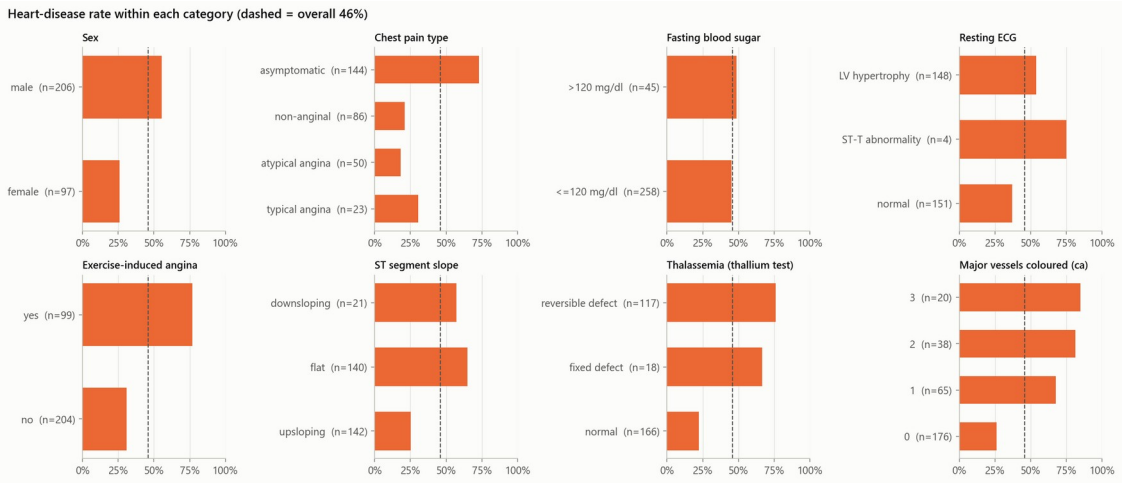


Figure 2. *Disease rate within each category; the dashed line is the overall rate. Bars far from it carry signal.*

- **Strongest signals:** reversible thallium defect (76% disease vs 22% for normal), number of coloured vessels (26% at 0 rising to 85% at 3), asymptomatic chest pain (73%, a known referral effect in this cohort) and exercise-induced angina (77% vs 31%).
- **Weak signals:** fasting blood sugar, cholesterol and resting blood pressure ($|\rho| \le 0.13$ with the target).
- **Redundancy is mild:** the strongest feature pair is `slope`–`oldpeak` ($\rho = 0.59$), so all features are kept. `thalach` falls with age ($\rho = -0.39$), which motivated an age-adjusted heart-rate feature (§5).
- **Encoding:** `cp`, `thal`, `restecg` and `slope` are nominal codes and are one-hot encoded; `restecg = 1` has only 4 patients, so the encoder must tolerate categories missing from a CV fold.

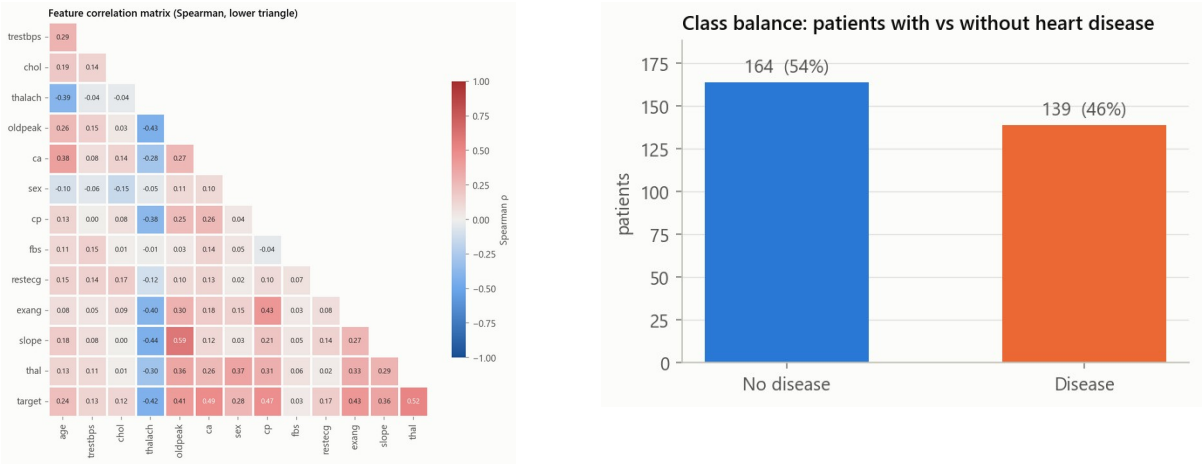


Figure 3. *Spearman correlation matrix (left) and class balance (right).*

5 Feature engineering and model development

5.1 Pipeline

The complete transformation is a single scikit-learn `Pipeline`, saved as one object, so training and serving cannot diverge:

- `FeatureEngineer` selects the 13 inputs, coerces them to float and optionally adds `hr_pct_max = thalach / (220 − age)`, the share of the age-predicted maximum heart rate reached.
- `ColumnTransformer`: numeric columns → median imputation → standard scaling; categorical codes → most-frequent imputation → one-hot encoding with `handle_unknown="ignore"`.
- Estimator: Logistic Regression, Random Forest or Gradient Boosting.

5.2 Tuning, evaluation and selection

A stratified 80/20 split holds out 61 patients that are used **only** for final reporting. Each model family is tuned with `GridSearchCV` (stratified 5-fold CV, scoring ROC-AUC, accuracy, precision, recall and F1) twice: with base features and with the engineered feature, which makes the feature an explicit ablation.

Model	Hyper-parameter grid	Candidates
Logistic Regression	C ∈ {0.01, 0.1, 1, 10} × penalty ∈ {L1, L2} × class_weight ∈ {none, balanced}	16
Random Forest	n_estimators ∈ {200, 500} × max_depth ∈ {none, 5, 8} × min_samples_leaf ∈ {1, 4} × max_features ∈ {sqrt, 0.5}	24
Gradient Boosting	n_estimators ∈ {100, 300} × learning_rate ∈ {0.03, 0.1} × max_depth ∈ {2, 3} × subsample ∈ {0.8, 1.0}	16

Table 3. *Search space: 56 candidates × 2 feature sets × 5 folds = 560 fits.*

> **Selection rule:** the champion has the highest mean CV ROC-AUC (threshold-independent). Candidates within 0.005 AUC of the best are treated as tied, and the tie is broken by CV recall, because a missed disease case costs more than a false alarm. The test set never influences selection.

Model	Features	CV ROC-AUC	CV recall	Test ROC-AUC	Test acc.	Test recall	Test F1
Logistic Regression	base	0.912 ± 0.017	0.792	0.962	0.885	0.929	0.881
Logistic Regression	+ hr_pct_max	0.909 ± 0.012	0.783	0.955	0.853	0.893	0.848
Random Forest	base	0.898 ± 0.029	0.774	0.957	0.885	0.929	0.881
Random Forest	+ hr_pct_max	0.897 ± 0.027	0.756	0.961	0.885	0.929	0.881
Gradient Boosting	base	0.884 ± 0.027	0.756	0.960	0.902	0.929	0.897
Gradient Boosting	+ hr_pct_max	0.883 ± 0.026	0.765	0.958	0.902	0.929	0.897

Table 4. *Best tuned candidate per model and feature set (champion highlighted).*

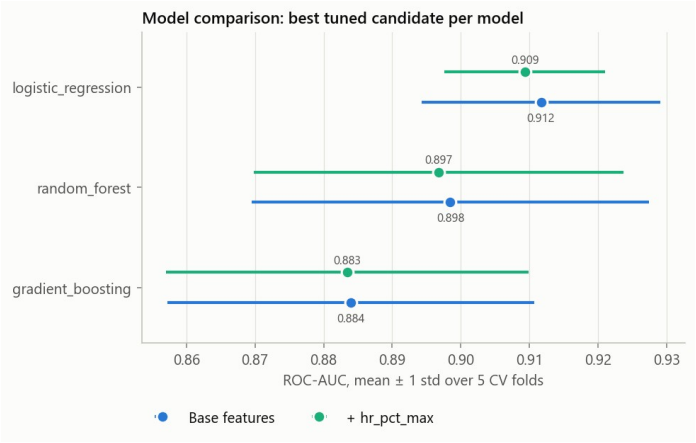


Figure 4. *CV ROC-AUC (mean ± 1 std): logistic regression leads; the engineered feature does not help.*

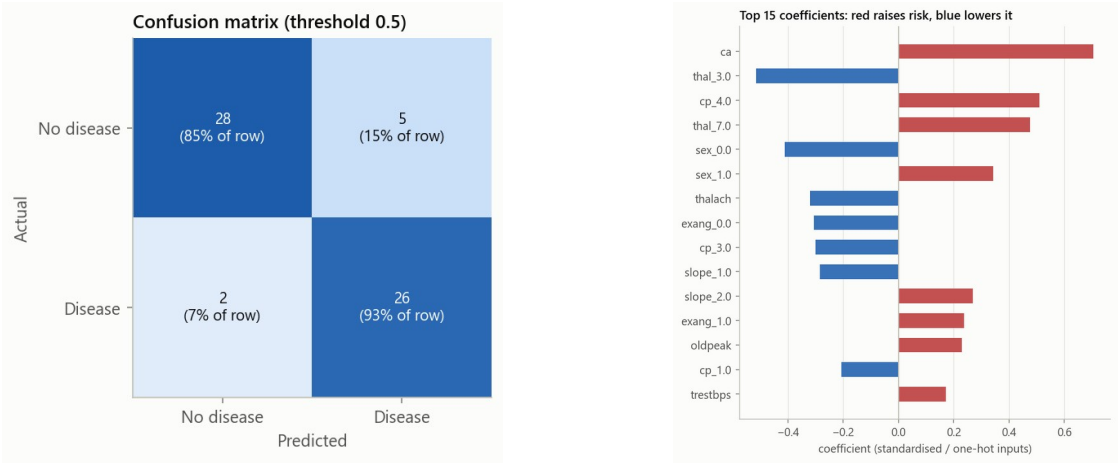


Figure 5. *Champion on the hold-out set: confusion matrix (left) and signed coefficients (right).*

- **The simplest model wins.** With 242 training rows the tree ensembles add variance without adding signal; Logistic Regression (C = 0.1) has the best and most stable CV AUC.
- **Negative ablation.** `hr_pct_max` changed CV AUC by −0.003 to 0.000 for every family, so the champion uses base features: `thalach` and `age` already carry the information.
- **Test scores exceed CV scores** for every model (≈ 0.96 vs 0.88–0.91). With 61 test patients the hold-out estimate is noisy; CV is the more reliable estimate, which is why selection uses it.
- **Coefficients agree with the EDA:** more coloured vessels, reversible defect and asymptomatic chest pain raise risk; a normal thallium result and a higher maximum heart rate lower it.

6 Experiment tracking (MLflow)

Tracking uses a SQLite backend (required for the model registry) with a local artefact store; both locations can be redirected by environment variables, which CI and tests use. The experiment `heart-disease-classification` currently holds **357 runs** from repeated full trainings.

- **One parent run per model family × feature set**, tagged with the data hash, git commit, scikit-learn version and split sizes. It logs the best parameters, CV mean and std for five metrics, hold-out metrics, ROC, confusion-matrix and feature-importance plots, the full `cv_results.csv`, and the fitted pipeline as an MLflow model with signature and input example.
- **One nested run per grid candidate** (112 per training) with its parameters, CV metrics and rank, so every configuration is traceable.
- **A `model-selection` run** stores the comparison table and plots and records the selection rule. The champion is registered as `heart-disease-classifier` and given the `@champion` alias.

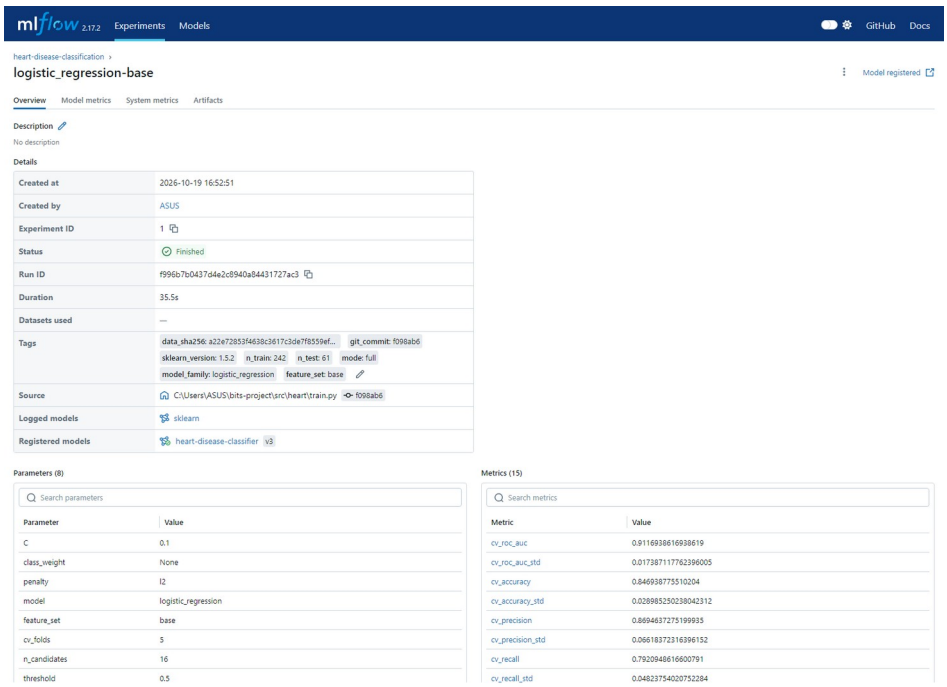


Figure 6. *MLflow run page of the champion: tags (data hash, git commit), parameters and metrics.*

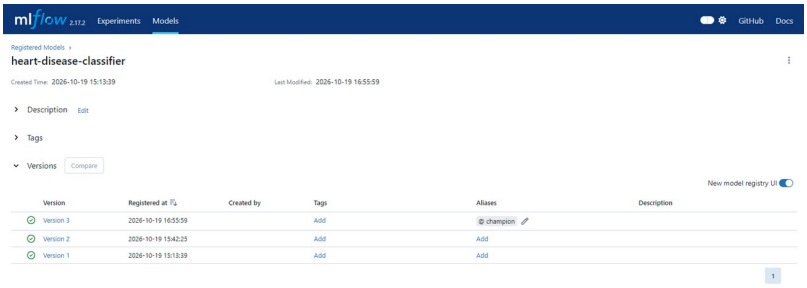


Figure 7. *Model registry: version 3 carries the @champion alias.*

7 Model packaging and reproducibility

Artefact	Format	Used by
`models/model.joblib`	Fitted sklearn Pipeline (preprocessing + model)	API, CLI (`scripts/predict.py`)
`models/mlflow_model/`	MLflow pyfunc model with the `heart` package bundled (`code_paths`), signature, input example, pinned requirements	`mlflow.pyfunc.load_model`, `mlflow models serve`
`models/metadata.json`	Model card: version, family, parameters, CV and test metrics, `model_sha256`, `data_sha256`, git commit	API `/health`, reproducibility tests, CI quality gate

Table 5. *Packaged artefacts.*

- **Pinned environment:** every package in `requirements.txt` is pinned with `==`, including transitive packages that broke builds when left floating (SQLAlchemy, Alembic, anyio, pyparsing). `requirements-serve.txt` is a subset for the container; a test enforces identical pins. `environment.yml` offers a conda alternative.
- **Determinism:** fixed seeds for the split, the CV folds and every estimator; retraining on the same platform reproduces `model.joblib` byte-for-byte.
- **Enforced by tests** (`test_reproducibility.py`): the model checksum matches the metadata, the training-data hash is unchanged, rebuilding the champion from `metadata.json` alone gives identical predictions, the recorded test metrics recompute exactly, and the MLflow model agrees with the joblib model.

- **Clean-setup proof:** the repository was copied to an empty folder, a new Python 3.11 venv was created and the full pipeline (install, lint, tests, download, train, predict) ran successfully; CI repeats this on Linux on every push.

> **Cross-platform finding.** The first Linux run of the CI jobs failed the data-hash test: pandas writes `\r\n` line endings on Windows and `\n` on Linux, so identical data hashed differently. The fix writes and hashes CSVs with LF everywhere. Pickled model bytes still differ between Windows and Linux while predictions and metrics are identical to 4 decimals, so cross-machine tests compare predictions, not bytes.

8 Serving API and containerisation

Endpoint	Purpose
`POST /predict`	One patient (JSON) → `prediction`, `risk_label`, `probability`, `confidence`, `model_version`, `request_id`
`POST /predict/batch`	1–100 patients in one call
`GET /health`	Liveness/readiness: model name, version, family, sha256, uptime
`GET /metrics`	Prometheus metrics (HTTP and model level)
`GET /docs`	OpenAPI / Swagger UI with real request and response examples

Table 6. *API endpoints.*

- **Validation:** Pydantic enforces UCI codes, clinical ranges and required fields and rejects unknown fields (422). `ca` and `thal` may be null and are imputed by the pipeline. `confidence` is the probability of the predicted class.
- **Safety:** at startup the API compares the model file's sha256 with `metadata.json` and refuses to serve a mismatched file. Unhandled errors return a generic 500 with the request ID; details go to the log only.
- **Image:** multi-stage build (dependencies in a builder venv, runtime copies only the venv, code and model); `.dockerignore` is an allow-list; serving dependencies only (no MLflow or plotting); pip and bundled test suites removed (771 → 604 MB); runs as uid 10001 with a `HEALTHCHECK`; uvicorn is PID 1, so `docker stop` shuts down gracefully in about 2.4 s.
- **Verification:** `scripts/smoke_test.py` runs 7 end-to-end checks (health, high- and low-risk predictions, request-ID echo, batch, invalid input → 422, metrics) and exits non-zero on any failure. It is used locally, in CI and after every Kubernetes deployment.

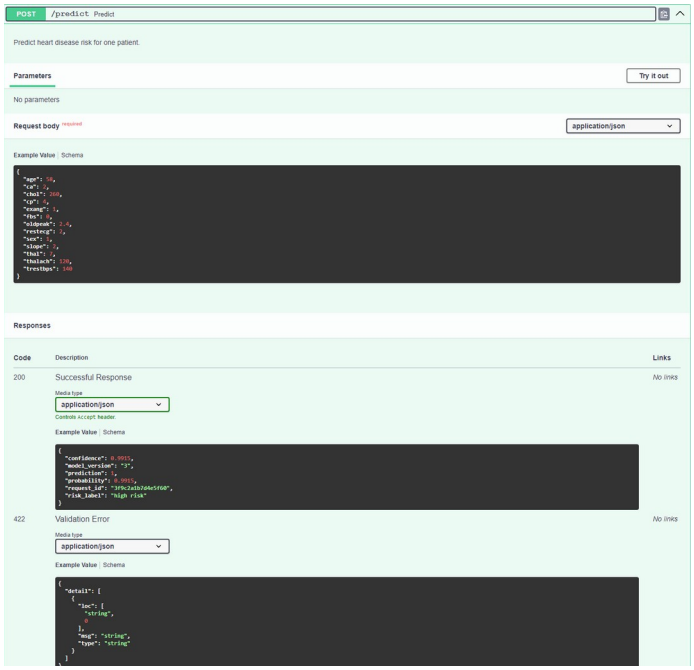


Figure 8. *Swagger UI for POST /predict with the documented request and response.*



Figure 9. *Container built and run locally, followed by the smoke test.*

9 CI/CD pipeline and automated testing

The workflow `.github/workflows/ci.yml` runs on every push and pull request. Jobs depend on each other, so any lint error, failing test, data mismatch, degraded model or broken container stops the pipeline. All steps run under `bash -eo pipefail`, so failures inside pipes are not hidden.

Job	Steps	Fails when	Outputs
lint	ruff (inline annotations), hadolint, kubeconform on both kustomizations, promtool on Prometheus config and rules	Any lint or schema error	Annotations
test	Pinned install, pip check, 131 pytest tests with coverage	A test fails or coverage < 85%	JUnit + coverage reports; coverage table in summary
train	Re-download UCI data and compare; full training with MLflow; quality gate vs committed model	Data differs, training error, test AUC drops > 0.02 or < 0.85	trained-model, mlflow-tracking artefacts; metrics table
docker	Build with the model trained in this run, run container, smoke test, push to GHCR (main only)	Build or any smoke check fails	Image `heart-api:latest`, `sha-<commit>`

Table 7. *CI/CD jobs.*

Tests (131, about 96% line coverage): data loading, cleaning and validation (19); features and preprocessing (12); training, selection, persistence and prediction (27); reproducibility of the committed artefacts (8); API behaviour, logs and metrics (32); Kubernetes and monitoring configuration (25); and the CI quality gate (8).

Tests were checked against deliberate defects (mutation checks): a wrong target threshold, a broken engineered-feature formula, a disabled recall tie-break, a wrong hyper-parameter in the metadata, a tampered model file and a misspelt dashboard metric were each caught by the intended test, and a typo in a Kubernetes field was rejected by the kubeconform CI check.

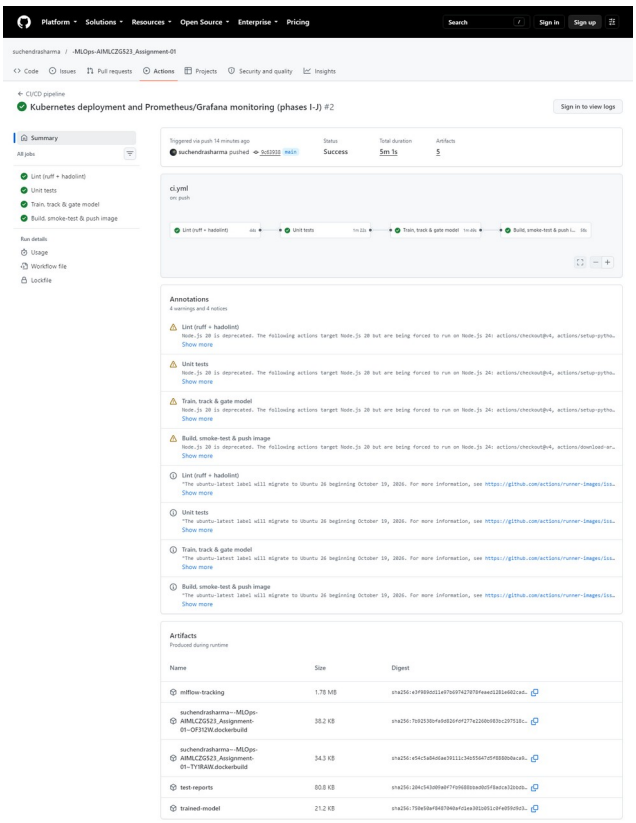


Figure 10. *Successful pipeline run: four dependent jobs and five uploaded artefacts.*

Screenshot to add
`screenshots/web_10_ci_failure.png`
Push a branch with a failing test, open a pull request and capture the red run.

Figure 11. *Pipeline stopping on a failing test (deliberately broken branch).*

10 Production deployment (Kubernetes)

The API runs on the Kubernetes cluster built into Docker Desktop, deployed with `kubectl apply -k k8s/` or `scripts/k8s_deploy.ps1`, which also builds the image, imports it into the cluster node, waits for the rollout and runs the smoke test. The current image is `heart-api:1.0.2`.

- **Deployment:** 2 replicas; rolling update with `maxUnavailable: 0`; startup, readiness and liveness probes on `/health`; CPU and memory requests and limits; non-root, read-only root filesystem, all capabilities dropped, seccomp `RuntimeDefault`; a 5 s `preStop` drain; Prometheus scrape annotations.
- **Service:** type `LoadBalancer` on port 80, which Docker Desktop binds to `localhost`. A PodDisruptionBudget keeps at least one pod during voluntary disruptions.
- **Validation:** manifests are schema-checked in CI (`kubeconform -strict`) and `test_k8s_manifests.py` enforces the rules above.

Check on the running cluster	Result
Rollout and smoke test via http://localhost	2/2 pods ready; 7/7 smoke checks pass
Load balancing (per connection)	Requests split 471 / 312 across the two pods
Self-healing (pod deleted)	Back to 2/2 ready in about 6 s
Rolling restart under load, no preStop hook	2 of 161 requests failed
Rolling restart under load, with preStop drain	0 failures in 4 of 5 rounds (~160 requests each)
15-minute load test incl. a live image upgrade	5400 requests: 5138 OK, 262 intended 422, **0 errors**

Table 8. *Deployment verification.*

```
(.venv) PS C:\Users\ASUS\bits-project> kubectl -n heart-mlops get all,pdb -o wide
NAME                             READY   STATUS    RESTARTS   AGE   IP            NODE                   NOMINATED NODE   READINESS GATES
pod/heart-api-5c958bbc9d-52gcn   1/1     Running   0          21m   10.244.0.30   desktop-control-plane   <none>           <none>
pod/heart-api-5c958bbc9d-6jzb9   1/1     Running   0          21m   10.244.0.31   desktop-control-plane   <none>           <none>

NAME                 TYPE           CLUSTER-IP     EXTERNAL-IP   PORT(S)        AGE     SELECTOR
service/heart-api    LoadBalancer   10.96.88.37    172.18.0.5    80:31358/TCP   5h21m   app.kubernetes.io/name=heart-api

NAME                        READY   UP-TO-DATE   AVAILABLE   AGE     CONTAINERS   IMAGES              SELECTOR
deployment.apps/heart-api   2/2     2            2           5h21m   heart-api    heart-api:1.0.2     app.kubernetes.io/name=heart-api

NAME                                   DESIRED   CURRENT   READY   AGE     CONTAINERS   IMAGES              SELECTOR
replicaset.apps/heart-api-5c6fdd988f   0         0         0       5h15m   heart-api    heart-api:1.0.0     app.kubernetes.io/name=heart-api,pod-template-hash=5c6fdd988f
replicaset.apps/heart-api-5c958bbc9d   2         2         2       21m     heart-api    heart-api:1.0.2     app.kubernetes.io/name=heart-api,pod-template-hash=5c958bbc9d
replicaset.apps/heart-api-6b69c98bd5   0         0         0       5h16m   heart-api    heart-api:1.0.0     app.kubernetes.io/name=heart-api,pod-template-hash=6b69c98bd5
replicaset.apps/heart-api-76d6f694b7   0         0         0       57m     heart-api    heart-api:1.0.1     app.kubernetes.io/name=heart-api,pod-template-hash=76d6f694b7
replicaset.apps/heart-api-c78fb6445    0         0         0       5h16m   heart-api    heart-api:1.0.0     app.kubernetes.io/name=heart-api,pod-template-hash=c78fb6445
replicaset.apps/heart-api-ffc9dd57d    0         0         0       62m     heart-api    heart-api:1.0.1     app.kubernetes.io/name=heart-api,pod-template-hash=ffc9dd57d

NAME                                     MIN AVAILABLE   MAX UNAVAILABLE   ALLOWED DISRUPTIONS   AGE
poddisruptionbudget.policy/heart-api     1               N/A               1                     5h21m
(.venv) PS C:\Users\ASUS\bits-project>
```

Figure 12. *Kubernetes resources: pods, Service with LoadBalancer, Deployment, ReplicaSet and PDB.*

```
(.venv) PS C:\Users\ASUS\bits-project> curl.exe -s -X POST http://localhost/predict -H "Content-Type: application/json" --data "@samples/patient_high_risk.json"
{"prediction":1,"risk_label":"high risk","probability":0.9915,"confidence":0.9915,"model_version":"3","request_id":"ba3a6485e37642c0"}
(.venv) PS C:\Users\ASUS\bits-project>
```

Figure 13. *Prediction served through the Kubernetes LoadBalancer.*

11 Monitoring and logging

- **Logging:** one JSON line per request (request ID, method, path, status, duration) and per prediction (label, probability); `X-Request-ID` is accepted or generated and echoed; probe endpoints log at DEBUG only.
- **Metrics:** HTTP request counts and latency histograms (buckets 5 ms – 2.5 s), `heart_predictions_total{risk_label}`, a probability histogram and a model-info gauge.
- **Prometheus** discovers the API pods through the Kubernetes API using the pod annotations; its RBAC is limited to reading pods in the API namespace. **Grafana** is provisioned with the data source and a 12-panel dashboard covering traffic, latency, errors, replicas, the serving model and model behaviour.

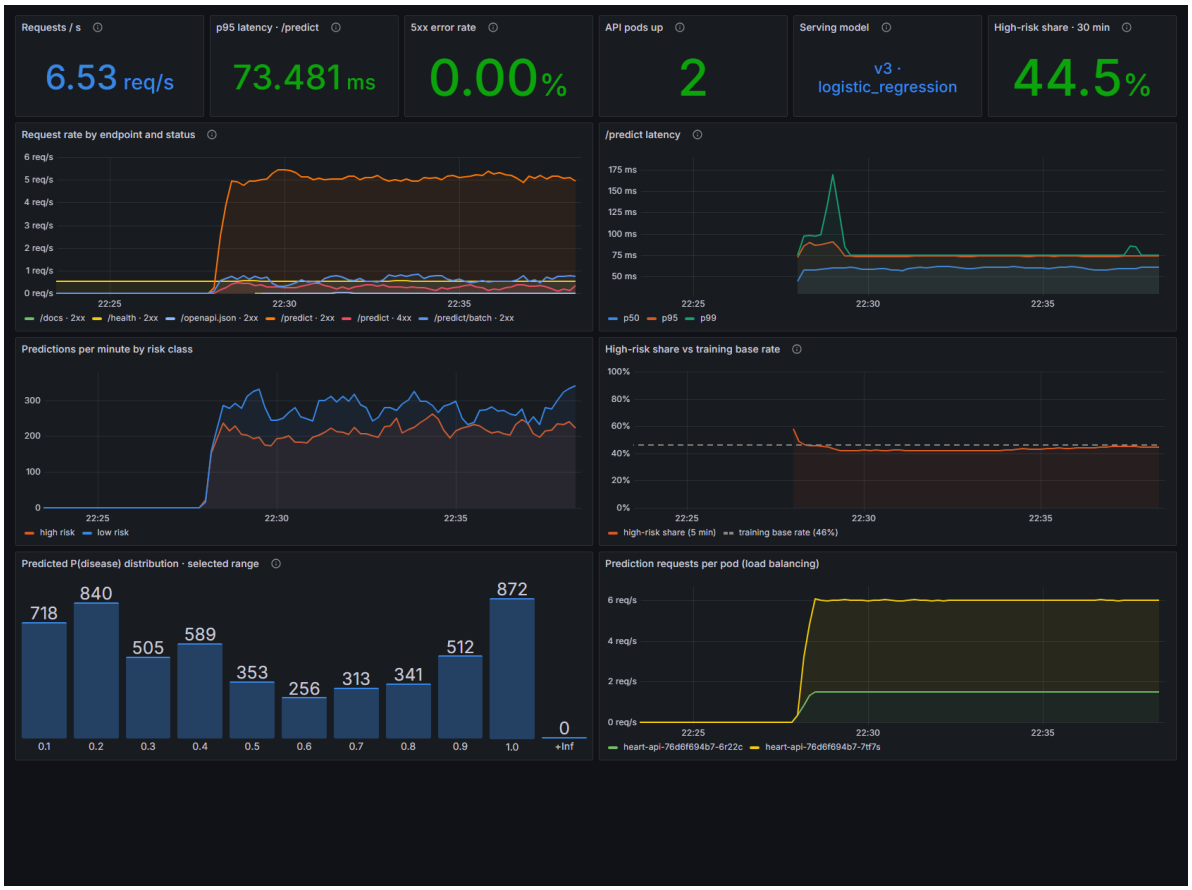


Figure 14. *Grafana dashboard under load: traffic, latency, errors, replicas, model version, risk-class mix, drift signal, probability distribution and per-pod load.*

Alert	Condition	Severity
HeartApiDown	No pod scraped successfully for 1 min	critical

Table 9. *Alert rules (validated with promtool).*

The alert pipeline was exercised end to end: scaling the API to one replica moved `HeartApiReplicasDegraded` to pending after 22 s and to firing after 142 s; it resolved 22 s after scaling back to two. `HeartPredictionDrift` is a simple model-behaviour monitor: a sustained change in the share of high-risk predictions indicates that the incoming population or the model has changed.

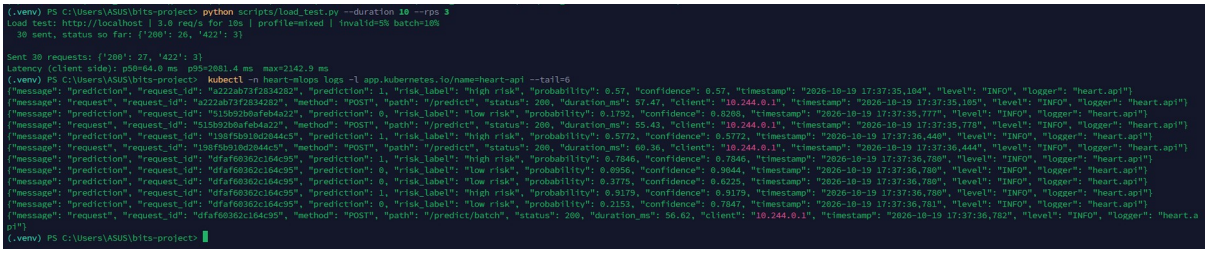


Figure 15. *Structured JSON logs from the API pods.*

11.1 Findings from load testing

- **Kubernetes balances connections, not requests.** A load generator sharing one keep-alive connection sent every request to a single pod; one connection per client thread split traffic 471 / 312.
- **Latency breakdown:** a prediction takes about 7 ms inside the pod and 8.8 ms through the Service inside the cluster. Requests through Docker Desktop's localhost LoadBalancer take about 60 ms; that delay is a local networking artefact and also appears in the server-side histogram, because the timer starts when headers arrive. The same forwarder occasionally stalls a connection for about 2 s, which shows up as client-side maxima near 2.1 s (and a high p95 in short runs) while server-side durations stay near 60 ms.
- **Histogram resolution matters:** the instrumentation library's default buckets (0.1, 0.5, 1 s) made every percentile of a 10 ms endpoint an interpolation artefact; explicit 5 ms – 2.5 s buckets fixed this.
- **Resource sizing:** a 500m CPU limit throttled about 12% of scheduling periods under load; a 1-CPU limit removed the throttling (it was not the cause of the latency above).

12 Challenges and lessons learned

Problem observed	Root cause	Resolution
MLflow failed to start	Unpinned transitive dependency: SQLAlchemy 2.1 removed a class MLflow 2.17 imports	Pinned SQLAlchemy and Alembic; all requirements pinned
Deprecation warnings; first pin rejected	Floating anyio / pyparsing; anyio 4.6.2 turned out to be a yanked, mistagged release	Pinned anyio 4.6.2.post1 and pyparsing 3.1.4
Data-hash test failed only on Linux	pandas uses `os.linesep` for CSV line endings	CSV written and hashed with LF on every OS
Pods could not find the image	Docker Desktop runs Kubernetes in kind mode with its own image store	Deploy script imports the image into the node; pinned tags
2 of 161 requests failed during a rollout	Terminating pod still received traffic until kube-proxy updated	5 s preStop drain; 0 failures in later rounds
Meaningless latency percentiles	Default histogram buckets too coarse for millisecond requests	Explicit 5 ms – 2.5 s buckets, locked by a test
All load on one pod	Services balance TCP connections; client reused one connection	One connection per load-generator thread
Inconsistent Swagger response example	Auto-generated example (prediction 0, "high risk")	Real examples from the champion model, checked by a test
CI deprecation warnings	GitHub actions v4/v5 run on Node 20	Moved to the lowest major versions that run on Node 24

Table 10. *Issues found during the build and how they were resolved.*

The common thread is that most problems were found by verification, not by inspection: cross-platform CI simulation, measuring real traffic during rollouts, reading the actual metric buckets and checking every screenshot. Each fix was locked in with a test or a CI check where possible.

13 Conclusion and future work

The project delivers a complete, reproducible path from raw UCI data to a monitored prediction service. A Logistic Regression pipeline (CV ROC-AUC 0.912, hold-out recall 0.929) is tracked in MLflow, packaged with checksums, verified by 131 automated tests and a CI quality gate, served by a hardened container on Kubernetes and observed through Prometheus and Grafana with service and model-level alerts.

Possible next steps:

- Deploy to a managed cluster (GKE/EKS/AKS) with an Ingress and TLS, pulling the image from GHCR; this would also remove the local LoadBalancer latency artefact.
- Input-drift monitoring (for example population stability index per feature) alongside the existing output-drift alert, with Alertmanager routing to e-mail or chat.
- Promotion workflow in the registry (challenger → champion) gated by the CI quality gate, plus a HorizontalPodAutoscaler backed by metrics-server.
- Probability calibration and threshold selection based on clinical cost, and evaluation on the other UCI sites as an external validation set.

Appendix A Repository structure and access

Path	Contents
`scripts/`	download_data.py, predict.py (CLI), smoke_test.py, load_test.py, check_model_quality.py, k8s_deploy.ps1
`src/heart/`	config, data, features, train, evaluate, predict, plotting
`api/`	FastAPI app, schemas, JSON logging
`data/processed/`	Cleaned dataset (raw download is reproducible and not committed)
`notebooks/`	01_eda.ipynb
`models/`	model.joblib, mlflow_model/, metadata.json
`tests/`	131 pytest tests in 11 files
`k8s/`, `k8s/monitoring/`	API manifests; Prometheus, alert rules, Grafana and dashboard
`.github/workflows/ci.yml`	CI/CD pipeline
`reports/`	This report (DOCX/PDF), figures, report build tooling
`screenshots/`	Evidence screenshots referenced in this report

Table 11. *Repository layout.*

Access (local deployment): after `scripts/k8s_deploy.ps1 -Monitoring`, the API is at http://localhost (docs at /docs), Prometheus at http://localhost:9090 and Grafana at http://localhost:3000 (anonymous read-only; admin login `admin` / `heart-mlops-demo` for local use only). The API is not deployed publicly.

Repository: https://github.com/suchendrasharma/-MLOps-AIMLCZG523_Assignment-01